

Intelligent Music Recommendation Engine

Technical Report

Cédric Verhaegh

Student Number: 221350

Applied Data Science & AI - Recommender Systems

June 2026

Abstract

This report presents the development, evaluation, and solution design of an intelligent music recommendation engine for a simulated major music streaming platform. The system uses implicit feedback from play counts and transforms it into rating-like preference scores for classical recommender-system algorithms. The project covers exploratory data analysis, sparsity reduction, outlier management, model development, multi-metric evaluation, and deployment strategy. Five recommendation strategies are compared: a popularity baseline, item-item k-nearest neighbours, singular value decomposition, Co-Clustering, and a weighted-average ensemble. The final evaluation shows that SVD is the strongest default personalized model by RMSE and AUC, while the weighted ensemble gives the best balance of Precision@10 and Recall@10. The final solution therefore uses SVD for default personalized recommendations, the popularity baseline for cold-start users, and the ensemble for batch-generated discovery playlists.

Contents

1	Introduction	2
2	Dataset and Problem Framing	2
3	Phase 1: Exploratory Data Analysis and Preprocessing	2
3.1	Distribution Analysis and Long-Tail Behaviour	2
3.2	Sparsity Reduction	3
3.3	Outlier Handling and Rating Transformation	4
4	Phase 2: Model Development	5
4.1	Popularity Baseline	5
4.2	Item-Item KNN Collaborative Filtering	5
4.3	SVD Matrix Factorization	5
4.4	Co-Clustering	5
4.5	Weighted-Average Ensemble	5
5	Phase 3: Evaluation and Metrics	6
5.1	Evaluation Setup	6
5.2	Predictive Accuracy: RMSE	6
5.3	Ranking Metrics	7
5.4	Classification Metrics: ROC and AUC	8
5.5	Final Model Interpretation	9
6	Phase 4: Solution Design and Strategy	9
6.1	Latency versus Accuracy	9
6.2	Deployment Strategy	10
6.3	Cold-Start Strategy	11
7	Limitations	11
8	Future Recommendations	12
8.1	Demographic and Contextual Data	12
8.2	Better Implicit-Feedback Modelling	12
8.3	Two-Tower Retrieval and Ranking Architecture	12
8.4	Monitoring and Fairness	12
9	Conclusion	12

1 Introduction

Digital music platforms contain very large catalogues, which makes manual discovery difficult for users. The business problem is therefore a classic recommendation problem: the system should predict user preferences and generate a personalized Top 10 playlist for existing users, while also handling users with zero listening history. This project uses a subset of the Million Song Dataset consisting of user-song play counts and song metadata.

The project brief states that the raw dataset contains approximately two million interactions, is highly sparse, and follows a long-tail distribution. It also requires four phases: exploratory data analysis and preprocessing, model development, evaluation and metrics, and solution design. The technical report must include performance visualizations, justification for the ensemble weights, and future recommendations such as demographic features or a deep learning two-tower architecture.

The implemented notebook follows this structure. The report summarizes the main findings and connects the practical choices to recommender-system concepts from the recommended resources, including matrix factorization (Koren et al., 2009), recommender evaluation (Shani & Gunawardana, 2011), and cold-start methods (Schein et al., 2002).

2 Dataset and Problem Framing

The interaction dataset contains 2,000,000 rows with the columns `user_id`, `song_id`, and `play_count`. The metadata file contains 1,000,000 rows with `song_id`, `title`, `release`, `artist_name`, and `year`. The interaction table contains 76,353 unique users and 10,000 unique interacted songs. All interacted songs have matching metadata, which allows recommendation examples to be displayed with readable titles and artists.

The feedback is implicit rather than explicit. A play count indicates that a user listened to a song, but it does not directly provide a star rating. This is important because many classical algorithms, including the Surprise implementations of KNN, SVD, and Co-Clustering, work with user-item-rating triples (Hug, 2020). For that reason, play counts were analysed as raw implicit feedback first, then transformed into bounded rating-like scores for model training.

3 Phase 1: Exploratory Data Analysis and Preprocessing

3.1 Distribution Analysis and Long-Tail Behaviour

The play-count distribution is strongly right-skewed. The median play count is 1, 75% of interactions have a play count of 3 or lower, and the maximum value is 2,213. The 99.9th percentile is 72, which shows that the maximum is far beyond normal listening behaviour. Figure 1 shows the log-transformed play-count distribution.

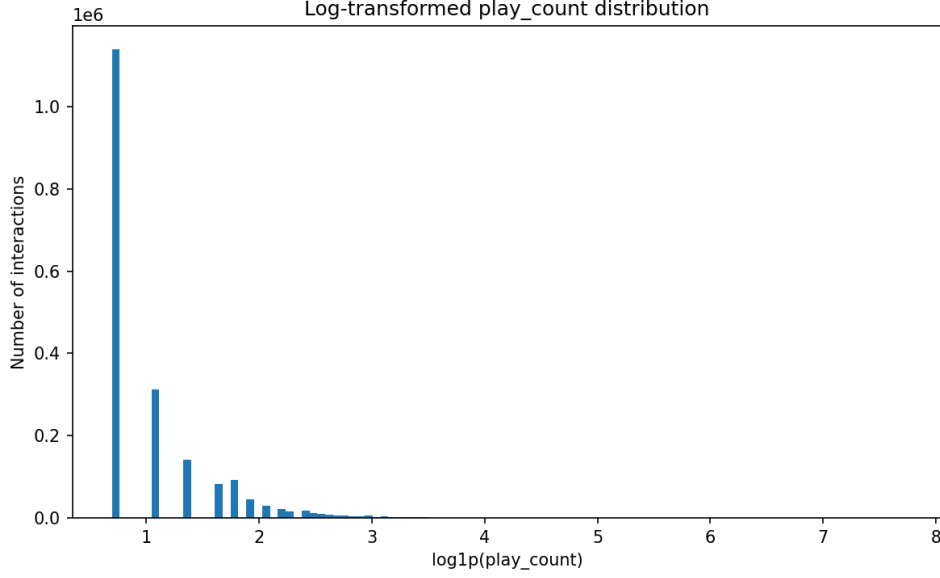


Figure 1: Log-transformed play-count distribution. The long tail is compressed but still visible.

The user and song distributions show the same pattern. The median user listened to 16 unique songs, while the most active user listened to 711 unique songs. The median song had 124 unique listeners, while the most popular song had 8,277 unique listeners. Figure 2 shows the long-tail song popularity curve. This confirms a key recommender-system risk: a purely popularity-based recommender can be strong as a baseline, but it may over-recommend mainstream songs and reduce discovery value for long-tail content.

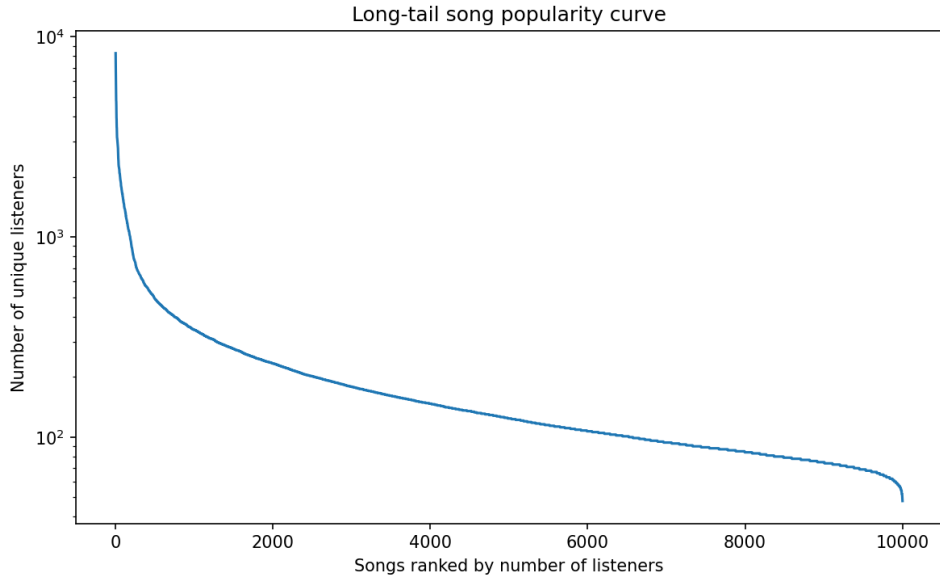


Figure 2: Long-tail song popularity curve based on unique listeners per song.

3.2 Sparsity Reduction

The raw user-song matrix has 76,353 users and 10,000 songs, which creates 763,530,000 possible user-song combinations. Only 2,000,000 combinations are observed. The density is therefore

approximately 0.26%, and the sparsity is approximately 99.74%. This confirms that missing user-song pairs should not be treated as dislikes. In most cases, missing interactions are unknown preferences.

Several filtering thresholds were tested. The selected full-data filtering strategy required at least 20 unique songs per user and at least 50 unique listeners per song. This produced a filtered full dataset with 1,504,680 interactions, 30,800 users, and 9,399 songs. For model comparison, a smaller reproducible user-sampled dataset was created with 97,215 interactions, 2,174 users, and 4,128 songs. User-level sampling was used because it preserves realistic user histories better than row-level sampling.

3.3 Outlier Handling and Rating Transformation

The modelling sample still contains extreme play counts. The 99th percentile play count is 25, while the maximum is 319. The preprocessing therefore clips play counts at 25, applies $\log_{1p}$, and scales the result into a bounded 1 to 5 preference score:

$$r_{ui} = 1 + 4 \times \frac{\log(1 + \min(c_{ui}, c_{0.99})) - \min(\log(1 + c))}{\max(\log(1 + c)) - \min(\log(1 + c))}, \quad (1)$$

where c_{ui} is the original play count and $c_{0.99}$ is the 99th percentile clipping threshold. This transformation keeps repeated listening as a stronger preference signal while reducing the influence of extreme outliers.

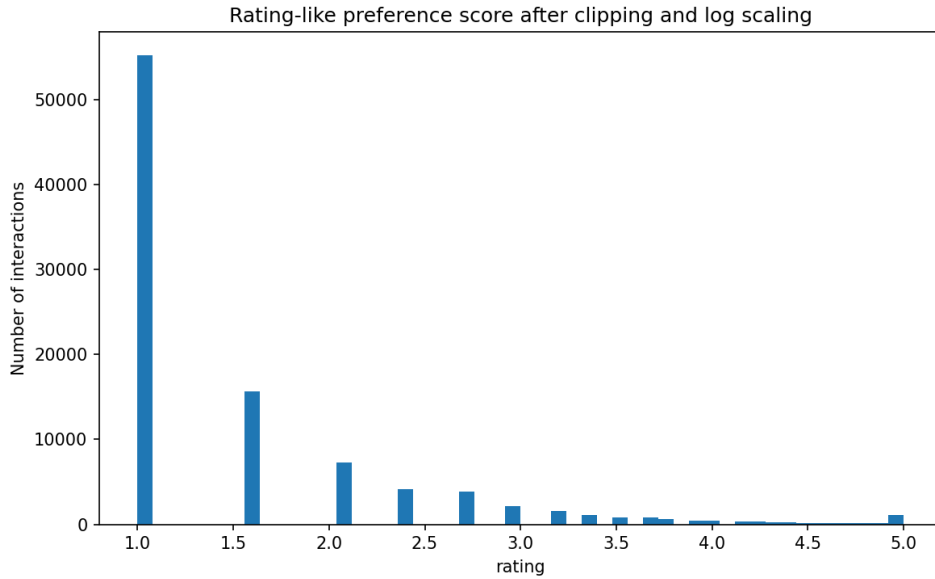


Figure 3: Rating-like preference score after clipping and log scaling.

A binary label was also created for classification-style evaluation. Interactions with at least two plays were labelled as `liked = 1`; single-play interactions were labelled as `liked = 0`. In the modelling sample, approximately 43% of interactions were labelled as repeated-listen positives, which is usable for ROC and AUC evaluation.

4 Phase 2: Model Development

The model-development phase implemented five recommendation strategies required by the project brief.

4.1 Popularity Baseline

The popularity baseline is a non-personalized recommender. It ranks songs using aggregate popularity signals such as number of users, total plays, and average rating-like preference. It provides two roles. First, it is a benchmark that more complex models should beat. Second, it is the cold-start fallback for users with no listening history.

4.2 Item-Item KNN Collaborative Filtering

The memory-based collaborative-filtering model uses item-item similarity. Item-item KNN was preferred over user-user KNN because the modelling sample contains fewer songs than users, and item similarity is often more stable when users have limited histories. KNN can be useful for finding songs similar to those a user already listened to, but it can become slower at inference time because it relies on neighbourhood-style similarity computations.

4.3 SVD Matrix Factorization

SVD was implemented as the model-based collaborative-filtering approach. Matrix factorization represents users and items through latent factors and predicts preferences by combining global effects, user and item biases, and latent user-item interactions (Koren et al., 2009). This is well suited to sparse recommendation data because the model can learn compressed latent structure from incomplete user-item matrices.

4.4 Co-Clustering

Co-Clustering was included as the unsupervised learning strategy. It groups users and songs simultaneously, which is conceptually related to biclustering approaches that cluster rows and columns of a data matrix together (Pedregosa et al., 2011). This can reveal broad interaction patterns. In this project it is useful as a required comparison model and as an analytical perspective, but it was not the strongest final recommender.

4.5 Weighted-Average Ensemble

The ensemble combines the predictions of the baseline, item-item KNN, SVD, and Co-Clustering models through a weighted average:

$$\hat{r}_{ui}^{ens} = 0.15\hat{r}_{ui}^{pop} + 0.25\hat{r}_{ui}^{knn} + 0.40\hat{r}_{ui}^{svd} + 0.20\hat{r}_{ui}^{co}. \quad (2)$$

These weights were chosen logically rather than tuned on the test set. SVD receives the highest weight because it had the strongest validation performance and is the most robust personalized model. KNN and Co-Clustering each receive moderate weights because they add

complementary neighbourhood and grouping signals. The popularity baseline receives a small weight because it provides a stable fallback signal, but too much popularity would increase popularity bias.

A neural-network wrapper was not used because the dataset favoured a transparent, reproducible, and explainable strategy. A neural wrapper would require additional hyperparameter tuning, more training time, and a larger validation process to avoid overfitting. The weighted-average ensemble is therefore more appropriate for this project scope.

5 Phase 3: Evaluation and Metrics

5.1 Evaluation Setup

The data was split into training, validation, and test sets. The validation set was used for model comparison and ensemble selection during Phase 2. The held-out test set was used only in Phase 3 for final evaluation. This avoids using the test set for model tuning.

The evaluation uses multiple metric groups, following the recommendation that recommender systems should not be judged by one metric only (Shani & Gunawardana, 2011). RMSE evaluates rating-prediction accuracy, ranking metrics evaluate Top-K recommendation quality, and ROC/AUC evaluates repeated-listen classification behaviour.

5.2 Predictive Accuracy: RMSE

The RMSE is calculated as:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (r_i - \hat{r}_i)^2}. \quad (3)$$

Table 1 and Figure 4 show the final test RMSE. SVD has the lowest RMSE, followed closely by the weighted ensemble.

Table 1: Final test RMSE by model.

Model	Test RMSE
SVD Matrix Factorization	0.8113
Weighted Ensemble	0.8142
Item-Item KNN	0.8562
Co-Clustering	0.8898
Popularity Baseline	0.9103

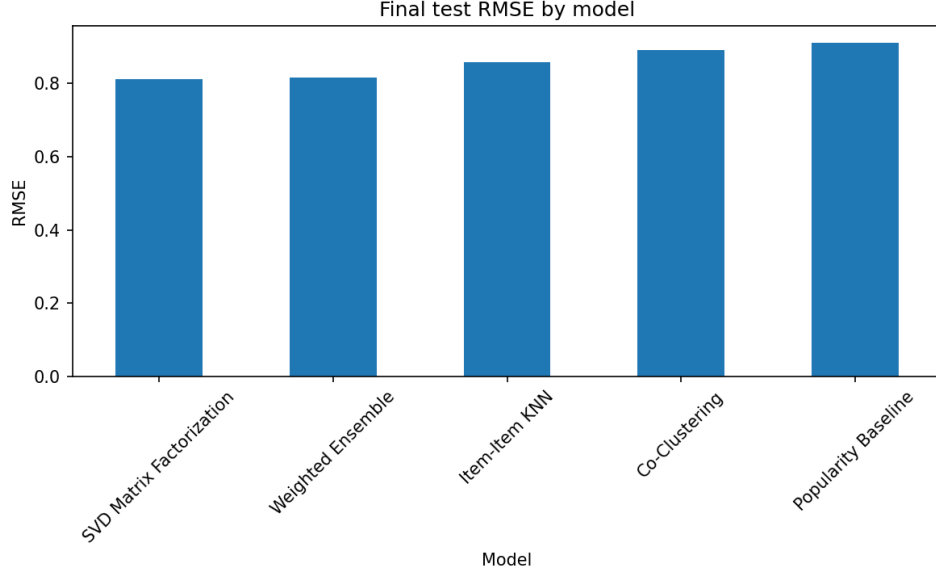


Figure 4: Final RMSE by model. Lower is better.

5.3 Ranking Metrics

Precision@K, Recall@K, and F1@K were calculated for K=10 and K=30. K=10 is especially relevant because the project objective is to generate a Top 10 playlist. At K=10, the weighted ensemble has the best F1@10, while item-item KNN has the highest Recall@10 by a very small margin. At K=30, recall is extremely high for all models, making K=30 less discriminative in this offline setup.

Table 2: Ranking metrics at K=10.

Model	Precision@10	Recall@10	F1@10
Weighted Ensemble	0.5358	0.9535	0.6463
Item-Item KNN	0.5354	0.9537	0.6460
SVD Matrix Factorization	0.5342	0.9507	0.6445
Co-Clustering	0.5337	0.9501	0.6440
Popularity Baseline	0.5337	0.9515	0.6440

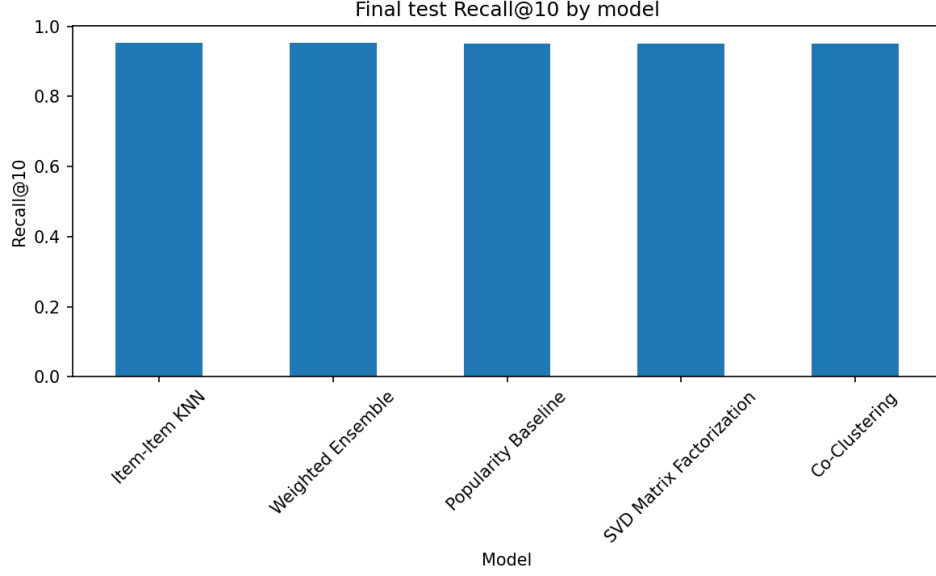


Figure 5: Recall@10 by model. Higher is better.

5.4 Classification Metrics: ROC and AUC

The ROC/AUC evaluation uses the binary repeated-listen label. It measures whether model scores can separate repeated-listen interactions from single-play interactions across decision thresholds. Figure 6 shows the ROC curves, and Table 3 shows the AUC scores. SVD achieves the highest AUC, followed by the weighted ensemble.

Table 3: Final AUC by model.

Model	AUC
SVD Matrix Factorization	0.7173
Weighted Ensemble	0.7147
Item-Item KNN	0.6833
Co-Clustering	0.6832
Popularity Baseline	0.5934

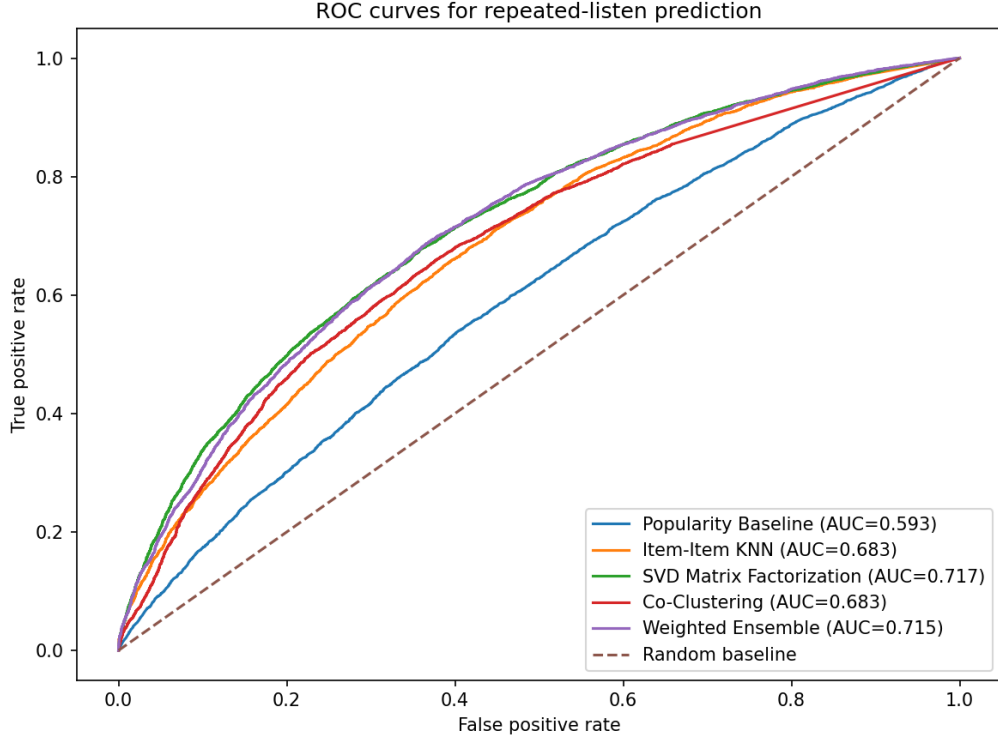


Figure 6: ROC curves for repeated-listen prediction.

5.5 Final Model Interpretation

The metrics favour slightly different models. SVD is best by RMSE and AUC, which makes it the safest final personalized model for accurate score prediction and repeated-listen discrimination. The weighted ensemble does not beat SVD on RMSE or AUC, but it achieves the best F1@10. Item-item KNN has the best Recall@10, but it is weaker than SVD by RMSE and AUC.

The final decision is therefore not based on one metric. SVD is selected as the default personalized recommendation model. The ensemble remains a useful advanced strategy when balanced Top 10 ranking quality is more important than latency. The popularity baseline remains the cold-start strategy.

6 Phase 4: Solution Design and Strategy

6.1 Latency versus Accuracy

Latency was measured on a sample of 1,000 test interactions with three timing repeats per model. Table 4 summarizes the results. The popularity baseline is fastest, followed by Co-Clustering and SVD. Item-item KNN and the weighted ensemble are slower. The ensemble is slower because it must calculate several component predictions before combining them.

Table 4: Inference latency comparison.

Model	ms per interaction	Interactions per second
Popularity Baseline	0.0004	2,254,283
Co-Clustering	0.0024	415,910
SVD Matrix Factorization	0.0032	309,262
Item-Item KNN	0.0279	35,847
Weighted Ensemble	0.0341	29,360

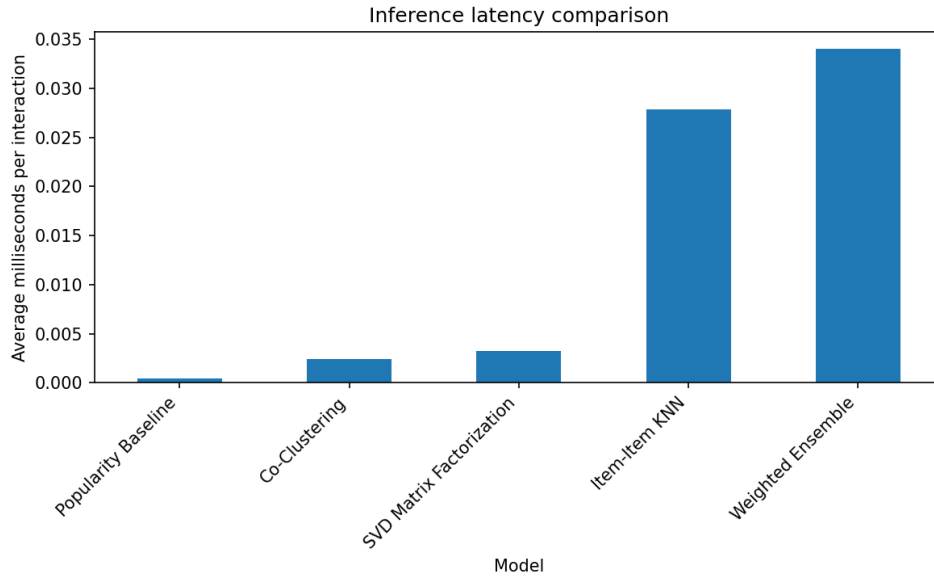


Figure 7: Average inference latency per interaction. Lower is better.

This creates a practical design trade-off. SVD gives the best overall accuracy and remains very fast, so it is suitable for real-time or cached personalized recommendations. The weighted ensemble is better positioned for batch-generated discovery playlists, where slightly higher inference cost is acceptable.

6.2 Deployment Strategy

The proposed deployment strategy is staged by use case:

Table 5: Recommended serving strategy by scenario.

Scenario	Recommended model	Reason
New user with zero listening history	Popularity Baseline	No user history is available, so collaborative filtering cannot personalize yet.
Default personalized recommendations	SVD Matrix Factorization	Best RMSE and AUC while remaining fast enough for real-time or cached serving.
Discovery-oriented Top 10 playlist	Weighted Ensemble	Best F1@10 and combines complementary model signals.
Recall-focused candidate recovery	Item-Item KNN	Strong Recall@10, useful for retrieving similar songs.
Catalogue analysis and segmentation	Co-Clustering	Useful for broad user-song grouping and offline insight.

6.3 Cold-Start Strategy

A user with zero listening history cannot be served effectively by pure collaborative filtering, because there is no user-specific behaviour to compare or factorize. This is the core cold-start issue described by Schein et al. (2002). The proposed cold-start solution is therefore to recommend a popularity-based Top 10 list first.

The staged cold-start strategy is:

1. **Zero interactions:** use the popularity baseline.
2. **A few interactions:** blend popularity with item-based similarity.
3. **Enough listening history:** switch to SVD as the default personalized model.
4. **Batch discovery playlists:** use the weighted ensemble when balanced Top 10 quality is more important than latency.

This strategy is practical because it begins with a robust non-personalized fallback and gradually increases personalization as behavioural evidence becomes available.

7 Limitations

Several limitations should be considered. First, the data is implicit feedback. A play count is a useful signal, but it is not the same as an explicit rating. A single play might mean mild interest, accidental exposure, or background listening. Second, filtering reduces sparsity and training time, but it also removes low-activity users and rare songs. This makes the model comparison more feasible, but it can underrepresent casual users and niche content. Third, the ranking evaluation ranks each user’s held-out test interactions rather than every unseen song in the full catalogue. This is computationally practical, but it is not the same as full production-scale top-N evaluation.

8 Future Recommendations

8.1 Demographic and Contextual Data

Future versions could integrate demographic or contextual information such as country, language, device type, time of day, playlist context, or genre metadata. This would help especially for cold-start situations where interaction history is limited.

8.2 Better Implicit-Feedback Modelling

The current project transforms play counts into rating-like values so that classical Surprise models can be used. A future system could use algorithms designed directly for implicit feedback, such as Bayesian Personalized Ranking (Rendle et al., 2012) or weighted matrix factorization for implicit-feedback datasets (Hu et al., 2008). These methods would better match the nature of play-count data because they model observed behaviour as implicit preference evidence rather than treating it as explicit rating information.

8.3 Two-Tower Retrieval and Ranking Architecture

For production-scale streaming platforms, a two-stage architecture would be more scalable. A two-tower retrieval model can generate candidate songs efficiently by embedding users and items in the same vector space, while a second ranking model can re-rank candidates using richer features. This aligns with the retrieval-versus-ranking approach described in modern recommendation-system guidance (Google Machine Learning Crash Course, 2025).

8.4 Monitoring and Fairness

A deployed recommender should be monitored for accuracy, latency, user engagement, popularity bias, catalogue coverage, and long-tail exposure. This is important because recommender systems can reinforce existing popularity patterns if they are optimized only for short-term engagement.

9 Conclusion

This project developed a complete intelligent music recommendation engine using implicit play-count data. The exploratory analysis confirmed the expected sparsity, long-tail distribution, popularity bias risk, and outlier problem. Preprocessing transformed play counts into bounded rating-like scores and created a repeated-listen label for classification evaluation.

Five models were implemented and compared: popularity baseline, item-item KNN, SVD, Co-Clustering, and a weighted ensemble. The final evaluation showed that SVD is the strongest model by RMSE and AUC, while the weighted ensemble gives the best F1@10. The deployment strategy therefore uses SVD as the default personalized recommender, the popularity baseline for cold-start users, and the ensemble for batch-generated discovery playlists. This satisfies the technical and implementation requirements of the project while leaving clear directions for future production improvements.

References

- Google Machine Learning Crash Course. (2025, August). *Recommendation systems* [Accessed for retrieval, ranking, and two-tower architecture concepts]. Retrieved June 2, 2026, from <https://developers.google.com/machine-learning/recommendation>
- Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. *2008 Eighth IEEE International Conference on Data Mining*, 263–272. <https://doi.org/10.1109/ICDM.2008.22>
- Hug, N. (2020). Surprise: A python library for recommender systems. *Journal of Open Source Software*, 5(52), 2174. <https://doi.org/10.21105/joss.02174>
- Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37. <https://doi.org/10.1109/MC.2009.263>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2012). BPR: Bayesian Personalized ranking from implicit feedback. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1205.2618>
- Schein, A. I., Popescul, A., Ungar, L. H., & Pennock, D. M. (2002). Methods and metrics for cold-start recommendations. *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 253–260. <https://doi.org/10.1145/564376.564421>
- Shani, G., & Gunawardana, A. (2011). Evaluating recommendation systems. In F. Ricci, L. Rokach, B. Shapira, & P. B. Kantor (Eds.), *Recommender systems handbook* (pp. 257–297). Springer US. https://doi.org/10.1007/978-0-387-85820-3_8